

Study Notes: Binary Trees & Binary Search Trees (BST)

1. Introduction: Why Trees?

Trees are the **most important data structure** for FAANG-level coding interviews. While Linked Lists use pointers to connect nodes linearly (one node to another), Trees branch out hierarchically (one node points to multiple nodes).

Why use Trees?

1. **Efficiency:** Operations like insertion, deletion, and searching take $O(\log N)$ time in balanced binary search trees, compared to $O(N)$ in Arrays or Linked Lists.
2. **Hierarchical Order:** Best for hierarchical data representation (e.g., File systems, organizational charts).
3. **Dynamic & Cost-Efficient:** They adapt dynamically (unlike sequential Arrays) without massive memory overheads for resizing.

Real-World Applications: File systems, Databases (fast retrieval), Network Routing algorithms, Decision Trees (Machine Learning), Huffman Coding (Compression), and building blocks for Graphs/Heaps.

2. Tree Terminology & Properties

Basic Terminology;

- **Node:** A single unit of data containing a value and pointers to other nodes.
- **Root Node:** The topmost node of the tree. Level 0.
- **Edge:** The connecting link between two nodes.
- **Child / Parent:** If node A points to node B, A is the parent, and B is the child.
- **Siblings:** Nodes that share the exact same parent.
- **Leaf Node:** A node with zero children (bottom-most nodes).
- **Ancestor / Descendant:** If there is a direct path from A to B going downward, A is an ancestor of B, and B is a descendant of A.

Height vs Level;

- **Height of a Node:** The maximum number of edges from that node down to a leaf node. (Height of a leaf node = 0).
- **Level of a Node:** The number of edges from the root node down to that node. (Level of the root node = 0).

3. Types of Binary Trees

A **Binary Tree** is a tree where every node has **at most 2 children** (0, 1, or 2).

1. **Complete Binary Tree:** All levels are completely filled, except possibly the last level. The last level *must* be filled strictly from **left to right**. (Used heavily in Heaps).
2. **Full / Strict Binary Tree:** Every node has either exactly **0 or 2 children**. Never 1 child.
3. **Perfect Binary Tree:** All internal nodes have 2 children, and ALL leaf nodes are on the exact same level. (Maximum possible nodes for a given height).

4. **Height-Balanced Binary Tree:** The height difference between the left and right subtrees of *any* node is at most 1 (≤ 1). Average height is $O(\log N)$. (e.g., AVL Trees, Red-Black Trees).
5. **Skewed Binary Tree:** Every node has only one child. It essentially devolves into a Linked List. (Worst-case time complexity: $O(N)$).
6. **Ordered Binary Tree:** Nodes follow a specific structural property regarding their values (e.g., Binary Search Tree).

4. Mathematical Properties (Crucial for Interviews)

- **Max Nodes in a Perfect Binary Tree of height H :** $2^{(H+1)} - 1$
- **Max Leaf Nodes in a Perfect Binary Tree of height H :** 2^H
- **Total Internal Nodes in a Perfect Binary Tree:** $\text{Total Nodes} - \text{Leaf Nodes} = (2^{(H+1)} - 1) - 2^H = 2^H - 1$
- **Minimum Levels for N Leaf Nodes:** $\log_2(N) + 1$
- **Property of Strict Binary Trees:** $\text{Total Leaf Nodes} = \text{Total Internal Nodes} + 1$. (Proof by induction: adding 2 children to a leaf adds 2 new leaves but converts the old leaf into an internal node. Net change: +1 leaf, +1 internal node).

5. Binary Search Trees (BST)

A Binary Search Tree is a Binary Tree with a strict ordering property:

- For **any** node, all values in its **Left Subtree** are *strictly smaller* than the node's value.
- For **any** node, all values in its **Right Subtree** are *strictly greater* than the node's value.

Insertion Logic $O(\log N)$

To insert a new value (e.g., 5):

1. Start at the Root.
2. If $5 < \text{Root}$, go to the Left Child.
3. If $5 > \text{Root}$, go to the Right Child.
4. Repeat this recursively until you hit **null**.
5. Create a new Node with 5 and link it at that **null** position.

Why is it $O(\log N)$? Because at every level, you are discarding half of the tree. You only make 1 comparison per level. The maximum number of comparisons equals the height of the tree, which is $\log(N)$ for a balanced tree.

Unbalanced BST Problem

If you insert sorted data (e.g., 1, 2, 3, 4, 5) into a standard BST, it will form a **Right-Skewed Tree**. The time complexity for searching/inserting degenerates to $O(N)$.

Fix: Use Self-Balancing Trees (AVL/Red-Black) or, if given a static sorted array, recursively pick the **middle element** to be the root, splitting the array in half.

6. Tree Traversals (The Most Important Concept)

Traversals determine the order in which we visit the nodes.

A. Depth-First Search (DFS) Traversals

1. Pre-Order Traversal (Node, Left, Right)

- **Rule:** Print the current Node -> Traverse Left completely -> Traverse Right completely.
- **Use Cases:** Used to create a copy of a tree, evaluate mathematical expressions, or serialize a tree into a string/array.

2. In-Order Traversal (Left, Node, Right)

- **Rule:** Traverse Left completely -> Print the current Node -> Traverse Right completely.
- **Use Cases: Crucial for BSTs!** An In-Order Traversal of a Binary Search Tree will *always* visit the nodes in **Ascending Sorted Order**.

3. Post-Order Traversal (Left, Right, Node)

- **Rule:** Traverse Left completely -> Traverse Right completely -> Print the current Node.
- **Use Cases:** Used for **Deleting** a tree (you must delete children before you delete the parent), bottom-up calculations like calculating the height or diameter of a tree.

B. Breadth-First Search (BFS) Traversal

- Traverses the tree level-by-level horizontally (Level 0, then Level 1, then Level 2).

7. Code Implementation Details (Java specific)

The Node Class;

A node is built using a custom class containing the value and two self-referential pointers.

Checking for a Balanced Tree

To check if a tree is balanced programmatically:

1. Ensure the tree is not null.
2. Get the height of the left child and right child.
3. Check `Math.abs(leftHeight - rightHeight) <= 1`.
4. Recursively check that the Left Subtree is balanced AND the Right Subtree is balanced.

Recursion is the Key

Trees and Recursion are inherently linked. Every Subtree is a Tree itself. You cannot master Tree problems without mastering Recursion. The call stack handles the "returning back up" the tree automatically during traversals.

Code and Implementation is in Code section